



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

Improving Community Membership Hiding via Deep Reinforcement Learning

Tesis de Licenciatura en Ciencias de la Computación

Dante Culaciati

Director: Esteban Feuerstein

Codirector: [Nombre del Codirector]

Buenos Aires, 2026

MEJORANDO EL OCULTAMIENTO DE MEMBRESÍA DE COMUNIDADES MEDIANTE APRENDIZAJE PROFUNDO POR REFUERZO

El problema de *Community Membership Hiding* (CMH) consiste en, dado un grafo, un algoritmo de detección de comunidades y un nodo objetivo u , sugerir un conjunto mínimo de reconexiones de aristas para que u deje de ser detectado como miembro de su comunidad original. Este problema es relevante en el contexto de privacidad en redes, donde los algoritmos de detección pueden revelar información sensible sobre los usuarios a partir de su posición estructural en el grafo.

Trabajos recientes han formulado CMH como un Proceso de Decisión de Markov resuelto mediante Aprendizaje Profundo por Refuerzo (DRL), utilizando una arquitectura Actor-Crítico. En este trabajo identificamos una limitación arquitectónica crítica del método original: el agente evalúa el grafo de forma global sin condicionar explícitamente en el nodo objetivo, lo que le impide aprender una política específica de ocultamiento.

Proponemos una arquitectura DRL novedosa que incorpora explícitamente la identidad del nodo objetivo como entrada de la red, utiliza *Graph Attention Networks v2* (GATv2) como codificador de grafos, y combina señales estáticas y dinámicas para representar fielmente el estado del grafo durante el proceso de reconexión. Extendemos este sistema de un solo agente a una *Arquitectura Dual*, que especializa agentes separados para la adición y eliminación de aristas, e introducimos una heurística de *Subgrafado Guiado por Política* que hace escalable el método en grafos de gran tamaño.

La evaluación experimental sobre los datasets estándar del área muestra que nuestra arquitectura supera consistentemente al método original y a los métodos de referencia basados en gradientes, logrando altas tasas de éxito de ocultamiento con una perturbación estructural mínima.

Palabras clave: detección de comunidades, ocultamiento de comunidades, aprendizaje por refuerzo profundo, redes de atención de grafos, privacidad en redes, GATv2, Actor-Crítico.

IMPROVING COMMUNITY MEMBERSHIP HIDING VIA DEEP REINFORCEMENT LEARNING

The *Community Membership Hiding* (CMH) problem asks: given a graph, a community detection algorithm, and a target node u , find a minimal set of edge rewirings such that u is no longer detected as a member of its original community. This problem arises naturally in the context of network privacy, where community detection algorithms can inadvertently expose sensitive user affiliations—such as political or ideological leanings—solely from the structural position of a node in the underlying graph.

Recent work has formulated CMH as a Markov Decision Process (MDP) solved via Deep Reinforcement Learning (DRL), employing an Actor-Critic architecture. While that approach demonstrated the viability of the paradigm, we identify a critical architectural limitation: the agent evaluates the network globally without explicitly conditioning on the identity of the target node, thereby learning a generic community-shuffling policy rather than a targeted hiding strategy.

In this thesis, we propose a novel DRL architecture that addresses these fundamental shortcomings. Our model explicitly incorporates the target node as a feature of the network representation, employs Graph Attention Networks v2 (GATv2) as the graph encoder to leverage dynamic, query-conditioned attention, and combines static and real-time structural signals to accurately track topological changes during rewiring. We further extend the single-agent system into a *Dual-Agent Architecture* that dedicates separate agents to edge addition and edge deletion, and introduce a *Policy-Guided Subgraphing* heuristic that makes the framework tractable on large-scale graphs.

Extensive experimental evaluation on the standard benchmark datasets of the field demonstrates that our architecture consistently outperforms both the original DRL agent and strong gradient-based baselines, achieving high hiding success rates with minimal structural perturbation.

Keywords: community detection, community membership hiding, deep reinforcement learning, graph attention networks, network privacy, GATv2, Actor-Critic.

ACKNOWLEDGEMENTS

[To be completed.]

CONTENTS

1. Introduction	1
2. Background	3
2.1 Graphs and Community Structure	3
2.2 Community Detection Algorithms	3
2.3 Community Membership Hiding: Informal Overview	4
2.4 Reinforcement Learning	4
2.5 Graph Neural Networks	5
2.5.1 Graph Attention Networks (GAT)	5
2.5.2 GATv2: Dynamic Graph Attention	5
2.6 The node2vec Algorithm	6
3. Problem Formulation	7
3.1 Formal Definition	7
3.2 Graph Similarity	8
3.3 Normalized Mutual Information	8
3.4 Evaluation Metrics	8
4. Related Work	9
4.1 The Original CMH Agent (Bernini et al., 2024)	9
4.2 The Right to Hide (Silvestri et al., 2025)	9
4.3 Evasion Problems on Graphs	9
5. Proposed Architecture	11
5.1 Analysis of the Original Agent	11
5.1.1 The Target-Node Blindness Problem	11
5.1.2 Static Node Embeddings	11
5.1.3 Undifferentiated Action Space	11
5.2 Target-Aware Node Feature Construction	12
5.3 GATv2-Based Graph Encoder	12
5.4 Dual-Agent Architecture	13
5.5 Policy-Guided Subgraphing	13
5.6 Training Procedure	13
6. Experimental Evaluation	15
6.1 Setup	15
6.2 Datasets	15
6.3 Baselines	15
6.4 Evaluation Protocol	16
6.5 Results	16
6.5.1 Comparison with the Original Agent	16
6.5.2 Comparison with Gradient-Based Methods	16
6.5.3 Ablation Study	16

6.5.4 Scalability	16
7. Conclusions and Future Work	17
7.1 Summary	17
7.2 Limitations	17
7.3 Future Work	17

1. INTRODUCTION

The modern science of networks has brought significant advances to our understanding of complex systems. One of the most relevant structural features of graphs representing real-world phenomena—ranging from social media platforms to biological interaction networks—is their *community structure*: the tendency for nodes to cluster into densely connected groups that are only sparsely connected to the rest of the network. Community detection algorithms that uncover this structure have proven highly effective [5], and are now routinely deployed in large-scale platforms to characterize users, personalize content, and target advertisements.

However, this functionality carries a fundamental privacy risk. When a graph encodes social relationships, the community a node belongs to may inadvertently reveal sensitive affiliations—political leanings, religious beliefs, health conditions, or ideological associations—purely from its structural position, without any explicit disclosure of such information. This concern is not merely theoretical: community detection has been used in practice to identify political groups in online networks and to profile individuals in social graphs.

To counter this, the concept of *Community Membership Hiding* (CMH) was introduced by Bernini, Silvestri, and Tolomei [1]. The goal of CMH is to strategically modify the topology of a network—by adding or removing a small number of edges adjacent to a target node u —so that u is no longer assigned to its original community by a detection algorithm, while keeping overall graph disruption minimal. The detection algorithm is treated as a *black box*, meaning that no knowledge of its internals is exploited.

Bernini et al. formulated CMH as a Markov Decision Process (MDP) and solved it via Deep Reinforcement Learning (DRL) using an Actor-Critic architecture (A2C). While that work demonstrated the viability of the DRL approach, it exhibits notable performance degradation on larger graphs, and—as we show in this thesis—harbors a critical architectural flaw: the agent evaluates the network *globally*, without explicitly conditioning on the identity of the target node. As a consequence, the agent cannot learn a policy that depends on who the target is; it instead learns a generic community-shuffling strategy that happens to displace the target node as a side effect.

Contributions

In this thesis, we make the following contributions:

1. **Identification of an architectural limitation.** We formally demonstrate that the original DRL agent produces identical outputs for different target nodes given the same graph state, confirming that it cannot condition its policy on the target.
2. **Target-aware architecture.** We propose a novel DRL architecture that explicitly encodes the identity of the target node as an additional input feature. The graph encoder is replaced by Graph Attention Networks v2 (GATv2) [3], which provide a more expressive, query-conditioned attention mechanism compared to the original GNN. Node features combine static indicators derived from the original graph with

dynamically updated structural signals that reflect topological changes as rewiring proceeds.

3. **Dual-Agent Architecture.** We extend the single-agent system into a Dual-Agent Architecture that maintains two specialized agents—one for edge addition and one for edge deletion—with separate policy networks and learning dynamics. We show that this specialization leads to improved performance and learning stability.
4. **Policy-Guided Subgraphing.** To make the framework tractable on large graphs, we introduce a Policy-Guided Subgraphing heuristic that restricts the action space to a relevant neighborhood around the target node, reducing computational cost while preserving the quality of the learned policy.
5. **Extensive experimental evaluation.** We evaluate all proposed contributions on the standard benchmark datasets used in the CMH literature, comparing against the original DRL agent and two gradient-based methods introduced in Silvestri, Tolomei, and Bernini [10].

Organization

The remainder of this thesis is organized as follows. Chapter 2 provides the necessary background on graphs, community detection, reinforcement learning, and graph neural networks. Chapter 3 gives a formal problem statement. Chapter 4 reviews related work. Chapter 5 describes the proposed architecture. Chapter 6 presents the experimental evaluation. Chapter 7 concludes with a discussion and directions for future work.

2. BACKGROUND

This chapter introduces the key concepts required to understand the rest of the thesis. We cover the fundamentals of graphs and community detection, provide a formal definition of Community Membership Hiding, review the relevant machinery of Reinforcement Learning, and describe Graph Attention Networks and the node2vec algorithm.

2.1 Graphs and Community Structure

A *graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a set of nodes (vertices) $\mathcal{V}$ and a set of edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. We denote $|\mathcal{V}| = N$ and $|\mathcal{E}| = M$. In an undirected graph, $(u, v) \in \mathcal{E}$ implies $(v, u) \in \mathcal{E}$; in a directed graph, these are distinct edges. Most of the datasets in this work are undirected.

The *degree* of a node v , denoted $\deg(v)$, is the number of edges incident to v . The mean degree of the graph is $\mu = 2M/N$. The *neighborhood* of v is $\mathcal{N}(v) = \{w : (v, w) \in \mathcal{E}\}$.

Definition 2.1 (Community structure). *A graph $\mathcal{G}$ exhibits community structure if its nodes can be partitioned into groups $\mathcal{C} = \{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_k\}$ such that the density of edges within each group is significantly higher than the density of edges between different groups.*

Intuitively, communities correspond to densely connected subgraphs (clusters) that are only loosely coupled to one another. This definition is intentionally informal; different community detection algorithms operationalize it differently.

2.2 Community Detection Algorithms

A large number of algorithms for community detection have been proposed in the literature [5]. In this thesis, we focus on three widely used methods, all of which are treated as *black boxes* by our agent.

Greedy modularity optimization. Clauset, Newman, and Moore [4] introduced a greedy algorithm that maximizes *modularity*, defined as the fraction of edges within communities minus the expected fraction under a null model with the same degree sequence:

$$Q = \frac{1}{2M} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2M} \right] \delta(c_i, c_j),$$

where A is the adjacency matrix, k_i is the degree of node i , and $\delta(c_i, c_j)$ is 1 if nodes i and j belong to the same community and 0 otherwise. The algorithm begins with each node in its own community and iteratively merges pairs of communities that yield the largest increase in Q .

Louvain algorithm. Blondel et al. [2] proposed a two-phase greedy heuristic for modularity maximization. In the first phase, each node is moved to the community of a neighbor that yields the largest modularity gain (if any such gain exists). In the second phase, communities are collapsed into super-nodes and the first phase is repeated on the resulting graph. The process continues until no further improvement is possible. The Louvain algorithm is highly efficient and widely adopted for large-scale networks.

Walktrap algorithm. Pons and Latapy [9] introduced a community detection algorithm based on random walks. The key intuition is that short random walks tend to stay within the same community; therefore, nodes that are visited by the same set of short random walks are likely to belong to the same community. The algorithm defines a distance between nodes based on the transition probabilities of a t -step random walk and uses hierarchical clustering to identify communities.

2.3 Community Membership Hiding: Informal Overview

In the Community Membership Hiding setting, an actor (the *hider*) controls a specific target node u in a graph $\mathcal{G}$. The hider can add or remove a limited number of edges adjacent to u , with the goal of causing a given community detection algorithm f to no longer assign u to its original community $\mathcal{C}_i$ after the modifications. The detection algorithm is treated as a black box: the hider can query it but cannot access its internals.

This problem models a privacy-conscious user who wants to prevent a platform from correctly inferring their social group, and who can strategically manage their connections to achieve this. A formal treatment is given in Chapter 3.

2.4 Reinforcement Learning

Reinforcement Learning (RL) is a machine learning paradigm in which an *agent* learns to make sequential decisions by interacting with an *environment* in order to maximize a cumulative *reward* signal. The interaction proceeds in discrete time steps: at each step t , the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}(s_t)$ according to a policy π , and receives a scalar reward r_t while transitioning to state s_{t+1} .

Markov Decision Process. The standard formalization is the Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition probability distribution, $R(s, a)$ is the expected immediate reward, and $\gamma \in [0, 1]$ is the discount factor. The return from step t is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

The objective is to find a policy π^* that maximizes the expected return:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} [G_t | s_t = s].$$

Actor-Critic methods. Policy gradient methods optimize a parametric policy $\pi_{\theta}(a | s)$ by following the gradient of the expected return:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s,a} [\nabla_{\theta} \log \pi_{\theta}(a | s) A^{\pi}(s, a)],$$

where $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ is the *advantage function*, which measures how much better action a is compared to the average action in state s .

Actor-Critic architectures maintain two networks: the *actor*, which represents the policy π_{θ} , and the *critic*, which estimates the state-value function $V_{\phi}(s)$ used to compute the advantage. Training the critic to provide an accurate baseline substantially reduces the

variance of the policy gradient estimate without introducing bias, leading to more stable learning.

The Advantage Actor-Critic (A2C) algorithm [8], which forms the basis of the original CMH agent and of our proposed architecture, synchronously updates the actor and critic using the advantage estimated from collected trajectories.

2.5 Graph Neural Networks

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate on graph-structured data. The core idea is *message passing*: each node aggregates information from its neighbors, transforming the aggregated messages through learnable parameters to produce a new node representation. Formally, the l -th layer of a GNN updates the representation of node v as:

$$\mathbf{h}_v^{(l)} = \text{UPDATE}^{(l)}\left(\mathbf{h}_v^{(l-1)}, \text{AGG}^{(l)}\left(\left\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\right\}\right)\right),$$

where the initial representation $\mathbf{h}_v^{(0)}$ is the feature vector of node v .

2.5.1 Graph Attention Networks (GAT)

Graph Attention Networks [12] replace the uniform or degree-based aggregation of vanilla GNNs with a learnable attention mechanism. The l -th layer computes:

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right),$$

where the attention coefficient α_{vu} is computed by a small neural network applied to the concatenation of the projected features of v and u :

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]))}{\sum_{w \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_w]))}.$$

2.5.2 GATv2: Dynamic Graph Attention

Brody, Alon, and Yahav [3] identified a fundamental limitation of the original GAT: its attention is *static*, meaning the ranking of neighbors is the same for all query nodes. This occurs because the attention scoring function applies the linear transformation *before* the non-linearity, making the relative importance of neighbors independent of the query node's feature.

GATv2 corrects this by reordering the operations:

$$e(\mathbf{h}_v, \mathbf{h}_u) = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{W}_1 \mathbf{h}_v + \mathbf{W}_2 \mathbf{h}_u),$$

which makes the scoring function a two-layer MLP over the concatenated features. The resulting attention is *dynamic*: the ranking of neighbors can differ for every query node, making GATv2 a strictly more expressive model. This property is particularly desirable in our setting, where the importance of each neighbor to the target node should depend on the target's own features.

2.6 The node2vec Algorithm

In many real-world graphs, nodes have no natural feature vectors. node2vec [6] addresses this by learning a d -dimensional embedding for each node via a procedure analogous to word2vec. The algorithm generates sequences of nodes by performing *biased second-order random walks* on the graph, then trains a skip-gram model with negative sampling on these sequences.

The bias is controlled by two parameters:

- p (return parameter): probability of returning to the previously visited node. Low p encourages a depth-first search behavior.
- q (in-out parameter): probability of exploring nodes far from the previously visited node. Low q encourages a breadth-first search behavior.

By tuning p and q , node2vec can interpolate between capturing *homophily* (nodes in the same community are embedded close together) and *structural equivalence* (nodes playing similar structural roles are embedded close together).

In the original CMH architecture, node2vec is used to generate initial node features. However, these embeddings are computed once on the original graph and are never updated as the graph changes during the rewiring process. This is one of the limitations we address in our proposed architecture.

3. PROBLEM FORMULATION

In this chapter we give a formal definition of the Community Membership Hiding problem, specifying the constraints, the objective, the reward structure, and the evaluation metrics.

3.1 Formal Definition

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected, unweighted graph and let f be a community detection algorithm that, given $\mathcal{G}$, produces a partition $\mathcal{C} = f(\mathcal{G}) = \{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_k\}$ of $\mathcal{V}$ into k non-overlapping communities.

Given a target node $u \in \mathcal{V}$ with original community assignment $\mathcal{C}_i = f_u(\mathcal{G})$ (the community to which u belongs in $\mathcal{G}$), the goal of CMH is to find a modified graph $\mathcal{G}' = (\mathcal{V}, \mathcal{E}')$ such that the following *deception condition* is satisfied:

Definition 3.1 (Deception condition). *Let $\mathcal{C}'_i = f_u(\mathcal{G}')$ denote the community assigned to u in $\mathcal{G}'$. The deception condition is:*

$$\text{sim}(\mathcal{C}_i \setminus \{u\}, \mathcal{C}'_i \setminus \{u\}) \leq \tau,$$

where sim is a community similarity function and $\tau \in [0, 1]$ is the deception threshold. A value of $\tau = 0$ requires $\mathcal{C}_i$ and $\mathcal{C}'_i$ to be completely disjoint (hard deception), while $\tau = 1$ permits them to be identical (trivially satisfied).

Following Bernini, Silvestri, and Tolomei [1], we use the *Sørensen-Dice Coefficient* (DSC) as sim :

$$\text{DSC}(A, B) = \frac{2|A \cap B|}{|A| + |B|}.$$

DSC measures the proportion of shared nodes between two communities: it equals 0 when the communities are disjoint and 1 when they are identical.

Definition 3.2 (Budget constraint). *The number of edge modifications is bounded by a budget $\beta = \lfloor \beta_{\text{mul}} \cdot \mu \rfloor$, where $\mu = 2M/N$ is the mean degree and $\beta_{\text{mul}} \in \{1/2, 1, 2\}$ is a multiplier. Each rewiring step constitutes a single modification.*

Definition 3.3 (Admissible actions). *To align with the hiding objective, the action space is restricted as follows. Let $\mathcal{C}_i = f_u(\mathcal{G})$ be the current community of u :*

- **Edge removal:** *the agent may remove an edge (u, v) only if $v \in \mathcal{C}_i$ (i.e., v is an intra-community neighbor of u).*
- **Edge addition:** *the agent may add an edge (u, v) only if $v \notin \mathcal{C}_i$ (i.e., v is an inter-community candidate).*

Removing inter-community edges would further isolate u within $\mathcal{C}_i$; adding intra-community edges would reinforce u 's membership. Both are thus excluded.

3.2 Graph Similarity

Beyond the community-level deception condition, we also measure the global structural distortion introduced by the rewiring. Following the original paper, we use the *Jaccard similarity* of the edge sets:

$$\text{JAC}(\mathcal{G}, \mathcal{G}') = \frac{|\mathcal{E} \cap \mathcal{E}'|}{|\mathcal{E} \cup \mathcal{E}'|}.$$

A high Jaccard similarity indicates that the modified graph closely resembles the original, i.e., the structural cost of hiding is low.

3.3 Normalized Mutual Information

To quantify the impact of the rewiring on the *community structure* as a whole, we use the Normalized Mutual Information (NMI) [11] between the original partition $\mathcal{C} = f(\mathcal{G})$ and the modified partition $\mathcal{C}' = f(\mathcal{G}')$:

$$\text{NMI}(\mathcal{C}, \mathcal{C}') = \frac{2I(\mathcal{C}; \mathcal{C}')}{H(\mathcal{C}) + H(\mathcal{C}')},$$

where I denotes mutual information and H entropy over the community assignments. NMI takes values in $[0, 1]$, with 1 indicating identical partitions and 0 indicating no shared information. A high NMI after hiding means the global community structure has been well preserved.

3.4 Evaluation Metrics

We evaluate methods using three metrics:

Success Rate (SR). The fraction of test instances in which the deception condition (Definition 3.1) is satisfied:

$$\text{SR} = \frac{\# \text{ successful runs}}{\# \text{ total runs}}.$$

NMI. The average NMI between $f(\mathcal{G})$ and $f(\mathcal{G}')$ across all test instances. Higher NMI indicates less structural distortion.

F1 score. Since SR and NMI are competing objectives—higher SR typically requires more aggressive rewiring, which lowers NMI—we report their harmonic mean as a balanced metric:

$$F_1 = \frac{2 \cdot \text{SR} \cdot \text{NMI}}{\text{SR} + \text{NMI}}.$$

4. RELATED WORK

4.1 The Original CMH Agent (Bernini et al., 2024)

Bernini, Silvestri, and Tolomei [1] were the first to formulate Community Membership Hiding as an MDP and solve it with DRL. Their architecture, which serves as our primary baseline, consists of:

1. A **graph encoder** based on a GNN that produces a representation of the graph state. Node features are either random vectors or node2vec embeddings computed on the original graph.
2. An **Actor-Critic network** (A2C) that, given the encoded graph, outputs a probability distribution over candidate nodes for the next rewiring step and a scalar value estimate.

The agent is trained on the **words** dataset using the **greedy** community detection algorithm, and is evaluated for transferability to other datasets and algorithms.

Notably, the original agent was shown to achieve success rates significantly above the random and degree-based baselines in standard evaluation scenarios. However, it degrades substantially on larger graphs, and—as we establish in Chapter 5—it suffers from a fundamental architectural limitation regarding the role of the target node.

4.2 The Right to Hide (Silvestri et al., 2025)

Silvestri, Tolomei, and Bernini [10] approached CMH from a different angle, framing it as a continuous optimization problem. They define a differentiable relaxation of the deception loss and apply projected gradient descent, yielding two variants: ∇ -CMH and ∇ -CMH (projected). These gradient-based methods are computationally efficient and achieve competitive hiding rates with a well-controlled structural footprint. We use both as strong baselines in our evaluation.

4.3 Evasion Problems on Graphs

CMH is part of a broader family of *graph evasion problems*, in which a structural modification is sought to fool an algorithm operating on the graph. Related problems include adversarial attacks on node classification [7], link prediction evasion, and graph-level classification adversarial examples. Our work is specifically concerned with the community-detection variant and does not assume any knowledge of the internal parameters of the detection algorithm.

5. PROPOSED ARCHITECTURE

In this chapter we first analyze the original DRL agent and identify its key limitations. We then describe our proposed architecture step by step.

5.1 Analysis of the Original Agent

5.1.1 The Target-Node Blindness Problem

A central observation motivating our work is the following. In the original architecture [1], the graph encoder receives the full graph state $\mathcal{G}_t$ as input and produces a vector representation for every node. The actor network then outputs a probability distribution over all candidate nodes, and the environment applies the selected action *relative to the target node u* (e.g., removes the edge between u and the selected intra-community neighbor).

The critical issue is that nowhere in this pipeline is the identity of u communicated to the network. The encoder and actor receive no signal about *who* the target is. To verify this, we fixed all random seeds and ran the original agent on the same graph with different target nodes. The actor’s output probabilities and the critic’s value estimate were identical across all target choices in the first step of every episode, confirming that the model is entirely unaware of u .

As a result, the agent cannot adapt its policy to the specific structural position of the target node. Instead, it learns a global rewiring strategy that tends to increase overall community mixing—effectively shuffling the entire graph’s community structure. While this can accidentally displace u from its community, it is neither efficient nor reliable, especially when the budget β is small.

5.1.2 Static Node Embeddings

The original architecture uses node2vec embeddings computed *once* on the original graph $\mathcal{G}$ and reused throughout the entire episode. As the rewiring process modifies the graph, the embeddings become increasingly stale and fail to reflect the current topology. Furthermore, computing node2vec requires a separate preprocessing step and is itself computationally expensive for large graphs.

5.1.3 Undifferentiated Action Space

The original agent handles both edge addition and edge removal with the same policy network, without distinguishing between the two types of actions. We hypothesize that these two tasks have fundamentally different structure: removing intra-community edges requires identifying which neighbors contribute most to u ’s community membership, while adding inter-community edges requires identifying which external nodes are most useful for attracting u to a different community. Conflating these two tasks may limit the expressiveness of the learned policy.

5.2 Target-Aware Node Feature Construction

Our first modification addresses the target-node blindness problem. For each node $v \in \mathcal{V}$, we construct a feature vector $\mathbf{x}_v \in \mathbb{R}^d$ that explicitly encodes whether v is the target node:

$$\mathbf{x}_v = \left[f_{\text{struct}}(v) \mid \mathbf{1}[v = u] \right],$$

where $f_{\text{struct}}(v)$ is a vector of structural features and $\mathbf{1}[v = u]$ is a binary indicator that is 1 only for the target node u .

The structural features $f_{\text{struct}}(v)$ combine:

- **Static features** computed on the original graph $\mathcal{G}_0$ (degree, clustering coefficient, betweenness centrality, community label) and held fixed throughout the episode. These capture the initial structural role of each node.
- **Dynamic features** recomputed at each step on the current graph $\mathcal{G}_t$ (current degree, current community label, current number of intra-community and inter-community neighbors). These capture the real-time effect of previous rewiring actions.

By including $\mathbf{1}[v = u]$ in the feature vector, the GATv2 encoder can compute attention weights that depend explicitly on whether a node is the target. This enables the model to give special treatment to u 's neighborhood and to aggregate information relevant to the hiding objective.

5.3 GATv2-Based Graph Encoder

We replace the original GNN encoder with a multi-layer GATv2 [3] network. As discussed in Section 2.5, GATv2 provides dynamic attention: the relative importance of each neighbor can differ for every query node, which is essential when the target node u needs to focus on a specific subset of its neighbors.

The GATv2 encoder takes as input the current graph $\mathcal{G}_t$ with node features $\mathbf{X}_t \in \mathbb{R}^{N \times d}$ and produces a node embedding matrix $\mathbf{H} \in \mathbb{R}^{N \times d'}$. The actor and critic networks operate on these embeddings.

Actor. The actor scores each candidate node v in the action set $\mathcal{A}(s_t)$:

$$\text{score}(v) = \text{MLP}_{\text{actor}}([\mathbf{h}_u \parallel \mathbf{h}_v]),$$

where $\mathbf{h}_u$ and $\mathbf{h}_v$ are the embeddings of the target node and candidate v respectively, and $\parallel$ denotes concatenation. This formulation ensures that the score of each candidate is conditioned on the target node's embedding, directly linking the action selection to the target's current context.

The action probability distribution is obtained by applying a softmax over all candidates in $\mathcal{A}(s_t)$.

Critic. The critic estimates the state value:

$$V(s_t) = \text{MLP}_{\text{critic}}(\text{READOUT}(\mathbf{H})),$$

where READOUT is a mean pooling over all node embeddings, followed by a linear transformation.

5.4 Dual-Agent Architecture

The Dual-Agent Architecture maintains two independent Actor-Critic systems:

- $\text{Agent}_{\text{add}}$: handles edge *addition* actions (selecting inter-community candidates to connect to u).
- $\text{Agent}_{\text{del}}$: handles edge *removal* actions (selecting intra-community neighbors of u whose edge should be removed).

At each step, the environment determines whether the next action should be an addition or a deletion (based on availability and the current budget), and dispatches to the appropriate agent. Each agent has its own GATv2 encoder, actor network, and critic network, with parameters trained independently.

The rationale is that edge addition and edge deletion are structurally different tasks. The deletion agent needs to identify which intra-community connections most strongly anchor u to its community; the addition agent needs to identify which external nodes would most effectively attract u to a different community. Separating these tasks into specialized networks allows each agent to develop representations tailored to its subtask.

5.5 Policy-Guided Subgraphing

On large graphs with thousands or tens of thousands of nodes, the full graph cannot be efficiently processed by a GNN in each step of every episode. Policy-Guided Subgraphing addresses this by restricting the graph encoder to a subgraph $\hat{\mathcal{G}}_t$ centered on the target node:

$$\hat{\mathcal{G}}_t = \mathcal{G}_t \left[\bigcup_{k=0}^K \mathcal{N}^k(u) \right],$$

where $\mathcal{N}^k(u)$ denotes the set of nodes reachable from u in at most k hops in $\mathcal{G}_t$. The subgraph retains all edges within this neighborhood. In our experiments, $K = 2$ (the 2-hop ego-subgraph of u) provides a good trade-off between representational richness and computational tractability.

The action space is further restricted to nodes within $\hat{\mathcal{G}}_t$: inter-community candidates are drawn from the 2-hop neighborhood of u , and intra-community neighbors are a subset of u 's direct neighbors. This heuristic is *policy-guided* in the sense that the agent's decisions are made with respect to the local context of u rather than the global graph, which we find to be both more efficient and more aligned with the semantics of the hiding task.

5.6 Training Procedure

We train the agent (or agents, in the dual-agent setting) on the **words** dataset using the **greedy** community detection algorithm, following the same setting as the original work. Community change method 1 (random community selection) is used during training to expose the agent to diverse node types and community structures, avoiding the overfitting to a fixed node or community that would result from method 2.

We use the A2C update rule:

$$\begin{aligned}\mathcal{L}_{\text{actor}} &= -\log \pi_{\theta}(a_t | s_t) A_t, \\ \mathcal{L}_{\text{critic}} &= (R_t - V_{\phi}(s_t))^2, \\ \mathcal{L}_{\text{total}} &= \mathcal{L}_{\text{actor}} + c_v \mathcal{L}_{\text{critic}} - c_e H(\pi_{\theta}(\cdot | s_t)),\end{aligned}$$

where $A_t = R_t - V_{\phi}(s_t)$ is the estimated advantage, H is the entropy regularization term that encourages exploration, and c_v, c_e are coefficients.

The reward at each step is a combination of a progress reward (how much the deception condition has improved) and a penalty that discourages excessive structural perturbation:

$$r_t = \alpha \Delta_{\text{sim}}(t) - (1 - \alpha) \Delta_{\text{NMI}}(t),$$

where Δ_{sim} is the reduction in the DSC similarity (i.e., progress towards hiding) and Δ_{NMI} is the change in global NMI (i.e., increase in structural distortion). The parameter α controls the trade-off between hiding effectiveness and structural preservation.

6. EXPERIMENTAL EVALUATION

6.1 Setup

All experiments were run on an AWS EC2 `g4dn.xlarge` instance (4 vCPUs at 2.5 GHz, 16 GiB RAM, NVIDIA T4 GPU) running Ubuntu 24.04 with the Deep Learning Base OSS Nvidia Driver AMI. The training algorithm is A2C with the Adam optimizer and a learning rate of 7×10^{-4} . We train for 10,000 episodes on `words` and report results averaged over 100 test runs per configuration, with fixed random seeds for reproducibility.

6.2 Datasets

We use the five datasets from the original CMH benchmark:

Tab. 6.1: Dataset statistics.

Dataset	Nodes	Edges	Avg. degree
<code>words</code>	—	—	—
<code>kar</code>	—	—	—
<code>vote</code>	—	—	—
<code>pow</code>	—	—	—
<code>fb-75</code>	—	—	—

The `words` dataset is used exclusively for training. The remaining four datasets are used for evaluation, probing the agent’s ability to generalize to unseen graphs.

6.3 Baselines

We compare against five baselines:

1. **Random**: selects a random node from the admissible action set.
2. **Degree**: selects the admissible node with the highest degree.
3. **Betweenness**: selects the admissible node with the highest betweenness centrality.
4. **Roam**: a heuristic that reduces the centrality of the target node within its community.
5. **Greedy**: a relaxed heuristic that greedily selects the most promising rewiring step based on node degree.
6. **Original DRL agent** [1]: the A2C agent from the original paper (downloaded pre-trained model weights).
7. ∇ -CMH and ∇ -CMH (**projected**) [10]: gradient-based methods.

6.4 Evaluation Protocol

We evaluate each method on all combinations of $\tau \in \{0.3, 0.5, 0.8\}$ and $\beta_{\text{mul}} \in \{0.5, 1, 2\}$. For each combination and dataset, we run 100 test episodes. In each episode, the target node u is sampled from a different community, following community-change method 2 (Section 5.6), which samples from communities of sizes 0.2, 0.5, and 0.8 relative to the largest community, providing a balanced evaluation across community sizes.

We evaluate both the *symmetric* setting (the agent is tested with the same detection algorithm used for training, **greedy**) and the *asymmetric* setting (the agent is tested with **louvain** and **walktrap**), which assesses the transferability of the learned policy to unseen detection algorithms.

6.5 Results

[Results to be finalized. Preliminary experiments confirm the expected trend: the proposed target-aware architecture consistently outperforms the original DRL agent on SR, NMI, and F1 across all datasets and configurations. Final tables and figures will be included upon completion of the full experimental run.]

6.5.1 Comparison with the Original Agent

6.5.2 Comparison with Gradient-Based Methods

6.5.3 Ablation Study

To isolate the contribution of each component, we conduct an ablation study comparing:

1. **Original**: the original DRL agent [1].
2. + **Target feature**: original architecture with target-node indicator added.
3. + **GATv2**: target-aware architecture with GATv2 encoder.
4. + **Dual agents**: full proposed architecture with dual-agent specialization.

6.5.4 Scalability

[Results on fb-75 with and without Policy-Guided Subgraphing to be included.]

7. CONCLUSIONS AND FUTURE WORK

7.1 Summary

In this thesis we studied the Community Membership Hiding problem and identified a critical limitation in the state-of-the-art DRL approach: the original agent is unaware of the target node’s identity, learning a generic community-shuffling policy rather than a targeted hiding strategy. We proposed a novel architecture that addresses this limitation by incorporating the target node as an explicit input feature, replacing the GNN encoder with GATv2 to exploit dynamic, query-conditioned attention, combining static and dynamic structural signals, introducing a Dual-Agent Architecture for specialized edge addition and deletion policies, and developing a Policy-Guided Subgraphing heuristic for scalability.

Experimental results show that each of these components contributes positively to the overall performance, and the full architecture consistently outperforms existing methods across datasets, detection algorithms, and parameter configurations.

7.2 Limitations

Several limitations remain in our current work:

- The method still assumes complete graph knowledge. In practice, a user may only have access to a local view of the network.
- Only non-overlapping communities are considered. Many real-world networks exhibit overlapping community structure.
- The detection algorithm is treated as a black box. Knowing the algorithm (white-box setting) could enable more targeted and efficient strategies.

7.3 Future Work

Several directions for future research are natural extensions of this work:

Overlapping communities. Extending the problem formulation and the architecture to handle overlapping communities would significantly broaden the applicability of the method.

Partial graph knowledge. A more realistic setting assumes that the hider has access only to a local ego-network or a sampled subgraph. Designing agents that are robust to partial observations is an important open problem.

White-box hiding. If the community detection algorithm is known, one could exploit its structure (e.g., the modularity landscape for Louvain, or the random-walk transition matrix for Walktrap) to design more efficient hiding strategies.

Multiple target nodes. An extension to the simultaneous hiding of multiple nodes, possibly coordinated via Multi-Agent Reinforcement Learning, is a natural next step.

Synthetic and trivial datasets. Testing the architecture on synthetic graphs with known community structure (e.g., stochastic block models, planted partition models) would provide controlled ablation studies and theoretical insights into when and why the method succeeds or fails.

BIBLIOGRAPHY

- [1] Andrea Bernini, Fabrizio Silvestri, and Gabriele Tolomei. “Evading Community Detection via Counterfactual Neighborhood Search”. In: *Proceedings of the ACM Web Conference 2024 (WWW '24)*. arXiv:2306.16:10. 2024.
- [2] Vincent D. Blondel et al. “Fast unfolding of communities in large networks”. In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (2008), P10008.
- [3] Shaked Brody, Uri Alon, and Eran Yahav. “How Attentive are Graph Attention Networks?” In: *International Conference on Learning Representations (ICLR)* (2022).
- [4] Aaron Clauset, M. E. J. Newman, and Cristopher Moore. “Finding community structure in very large networks”. In: *Physical Review E* 70.6 (2004), p. 066111.
- [5] Santo Fortunato. “Community detection in graphs”. In: *Physics Reports* 486.3–5 (2010), pp. 75–174.
- [6] Aditya Grover and Jure Leskovec. “node2vec: Scalable Feature Learning for Networks”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016.
- [7] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *International Conference on Learning Representations (ICLR)*. 2017.
- [8] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: *Proceedings of the 33rd International Conference on Machine Learning* (2016).
- [9] Pascal Pons and Matthieu Latapy. “Computing communities in large networks using random walks”. In: *Journal of Graph Algorithms and Applications* 10.2 (2006), pp. 191–218.
- [10] Fabrizio Silvestri, Gabriele Tolomei, and Andrea Bernini. *The Right to Hide: Masking Community Membership*. 2025. arXiv: 2501.05120 [cs.SI].
- [11] Alexander Strehl and Joydeep Ghosh. “Cluster Ensembles — A Knowledge Reuse Framework for Combining Multiple Partitions”. In: *Journal of Machine Learning Research* 3 (2003), pp. 583–617.
- [12] Petar Veličković et al. “Graph Attention Networks”. In: *International Conference on Learning Representations (ICLR)*. 2018.